[claude] AI 역량평가 Lv3 · 시험 직전 핵심팁

2026-11-07(토) 13:00–17:00 · 12:50 착석 · 학생증 | 순위: ● 1순위 = 0점·감점 방지 · ● 2순위 = 점수 올리기 · ● 3순위 = 여유 있을 때

코드는 ***_answer_optimized.ipynb** 기준입니다. 공식 예상문제 형식에 맞춘 것이므로, 실제 시험 문제가 다르면 **개념**만 가져가세요.

0. 공통 (모든 문항)

● 1순위

1. 첫 셀 `student_id = '001234'` : **따옴표 안에, 앞자리 0까지** 넣고 실행하세요. 이 값으로 제출 파일명이 자동으로 정해집니다.
2. 커널은 **그 폴더의** `.venv\Scripts\python.exe` 로 고르고, 첫 환경 셀에서 .venv와 GPU가 확인되는지 봅니다.
3. `# 수정 금지` 셀은 손대지 않습니다. `# 수정 가능` 셀이라도 **함수 이름, 반환 형태, 인자 이름**은 유지합니다.
4. 전처리나 모델을 바꾸면 **Dataset/DataLoader 셀과 모델 생성 셀을 다시 실행**합니다. 매번 사전학습 가중치에서 새로 시작해야 합니다.
5. 제출물은 **노트북(.ipynb) + `outputs/P*_<수험번호>.pth` + `_history.csv`** 입니다. **실제로 실행한 결과(출력)를 노트북에 남기고**, 출력을 지우지 마세요.

● 2순위

1. **과정 점수**: 바꾼 이유를 **주석 한 줄**이나 마크다운으로 적으세요. 예: `# train은 정면, val은 기울어짐 → 회전 증강`
2. **시간 배분**(4시간, 5문항): GPU 학습을 걸어 둔 동안 **P3(학습 없음, API만 사용)**나 다음 문항의 코드를 작성합니다. 노트북 두 개에서 동시에 GPU 학습을 돌리면 느려지거나 메모리가 부족할 수 있습니다.
3. 시간이 모자라면 **먼저 제출 가능한 상태를 만드세요**. 기본 코드로 한 번 끝까지 돌려 .pth와 history를 확보한 뒤 개선합니다.

● 3순위 · 자동완성

- `Ctrl+Space` 는 자동완성 제안, `Ctrl+Shift+Space` 는 함수 인자 힌트를 띄웁니다. `T.Ra` 를 입력하면 Random* 변환이 나오고, `torch.optim.lr_scheduler.On` 을 입력하면 OneCycleLR이 나옵니다.
- 인자 이름이 기억나지 않으면 **docs 폴더의 오프라인 문서**(`docs/index.html`)를 봅니다.

P1-1 클래스 불균형 · 오버샘플링 + 반복분 증강 (macro-F1)

● 1순위: `processing_training_data`

⚠ 직접 만든 PDF의 실수 4곳: **반복분의 flag는 `np.ones` (True)**입니다. zeros로 두면 증강이 적용되지 않습니다. loop는 **`range(len(counts))`** 로 돕니다. len(labels)는 틀립니다. 오타 `memebers` , `concantenate` 도 고치세요.

```
def processing_training_data(labels):
    labels = np.asarray(labels); rng = np.random.default_rng(seed=42)
    counts = np.bincount(labels); target = counts.max()      # 가장 많은 클래스 수에 맞춤
    indices, augment_flags = [], []
    for label in range(len(counts)):                          # ← 클래스 개수만큼
        members = np.flatnonzero(labels == label)
        if len(members) == 0: continue
        indices.append(members)
        augment_flags.append(np.zeros(len(members), dtype=bool))   # 원본 = False
        extra = target - len(members)
        if extra > 0:
            indices.append(rng.choice(members, size=extra, replace=True))
            augment_flags.append(np.ones(extra, dtype=bool))       # 반복분 = True
    indices = np.concatenate(indices).astype(int)
    augment_flags = np.concatenate(augment_flags).astype(bool)
    return indices, augment_flags                             # 두 배열 길이가 같아야 함
```

● 2순위: `train_transform` (flag가 True인 반복분에만 적용)

기본 train_transform은 eval_transform과 같아서 증강이 전혀 없습니다. 같은 이미지가 그대로 반복되면 과적합되므로 **약한 증강**을 넣습니다.

```
train_transform = T.Compose([
    T.ToPILImage(),                                  # 맨 앞 (입력이 numpy)
    T.RandomHorizontalFlip(p=0.5),
    T.RandomAffine(degrees=8, translate=(0.06, 0.06), scale=(0.92, 1.08), fill=255),
    T.ColorJitter(brightness=0.2, contrast=0.2),
    T.ToTensor(), T.Normalize(mean, std),            # 맨 뒤 (eval과 동일)
])
```

● 3순위

- CPU QUICK 3 epoch에서 val macro-F1이 0.727로, GPU 기준선 0.521보다 높게 나왔습니다. 원래 정답의 GPU 결과는 0.783입니다.

P1-2 얼굴 나이 예측 · 분포 차이(train 정면 ↔ val 카메라) (CS@10)

● 1순위: 진단 + 숫자 5개만 바꾸기

- **핵심은 분포 차이**입니다. train은 정돈된 정면 사진이고, val/test는 기울어지고 이동되고 확대된 카메라 사진입니다. 그래서 **val에서 보이는 만큼만** 증강합니다.
- 제공된 값은 **과한 증강**입니다. shift 0.80이면 얼굴이 화면 밖으로 나가고, shear 50°는 얼굴형을 망가뜨리고, 가림 확률이 95%입니다. 이 값들을 크게 줄입니다.
- 바꾸면 안 되는 것: 입력 크기 112, ToTensor/Normalize, 모델, 25 epoch, criterion.

변수	제공(과함)	정답	이유
`shift`	0.80	**0.12**	위치 이동 ±12%
`shear`	(50, 50)	**(-8.0, 8.0)**	약하게, 양방향
`tilt`	0.0	**30.0**	카메라 기울기 ±30° (val에서 확인)
`mask_probability`	0.95	**0.25**	가림은 가끔만
`mask_area`	(0.02, 0.12)	**(0.02, 0.08)**	작게

● 2순위: 변환 2개 추가 (나머지 순서는 그대로)

```
T.RandomAffine(degrees=0, translate=(shift, shift), scale=(1.0, 1.25), ...)   # 확대된 val 얼굴 대응
...
T.RandomRotation(degrees=tilt, ...),
T.ColorJitter(brightness=0.3, contrast=0.2),                                  # ToTensor 앞에
T.ToTensor(),
T.RandomErasing(p=mask_probability, scale=mask_area, ratio=(0.5, 2.0), value=mean),  # Tensor 뒤
T.Normalize(mean, std),
```

● 3순위: 과정 점수

- 확인용 그림 셀에 **val 이미지 줄을 추가**해 train과 나란히 보여 주세요. `plt.subplots(3, 5)` 로 1줄 train, 2줄 val, 3줄 증강 결과를 그리고, 마크다운에 진단을 한 줄 적습니다.

P2-1 학습 범위 제어(부분 freeze) + 직접 구현한 label smoothing loss

● 1순위: freeze + train() + forward

⚠ PDF의 실수: `x = self.stages[3]` 에는 **(x)**가 빠져 있습니다. head 앞의 **`x = x.mean(dim=(2, 3))`** 도 빠져 있습니다. 학습 범위(`requires_grad`)와 forward의 no_grad 범위가 **서로 맞아야** 합니다.

```
# __init__ 안 (가중치 로드·fc 교체 뒤)
self.requires_grad_(False)                # 일단 전부 freeze
self.stages[2:].requires_grad_(True)      # stage 2, 3 학습 (PDF엔 stages[2]만 → optimized는 2:)
self.head.requires_grad_(True)

def train(self, mode=True):
    not_train = [self.patch_embed, self.stages[0], self.stages[1]]
    super().train(mode)
    for module in not_train:
        module.train(False)               # freeze 부분은 항상 eval (BN·Dropout 고정)
    return self                           # ← 꼭 반환

def forward(self, x):
    with torch.no_grad():                 # freeze 부분은 기울기 계산 X
        x = self.patch_embed(x)
        x = self.stages[0](x)
        x = self.stages[1](x)
    x = self.stages[2](x)
    x = self.stages[3](x)
    x = x.mean(dim=(2, 3))                # ← (B,C,H,W) → (B,C)
    x = self.head.norm(x)
    return self.head.fc(x)
```

● 1순위: `custom_loss` (완성된 Loss API 금지 → CrossEntropyLoss, F.cross_entropy 사용 X)

공식: loss = (1−ε)·(−log p[정답]) + ε·(−mean$_k$ log p[k])

```
def custom_loss(logits, targets):
    epsilon = 0.1
    log_probs = F.log_softmax(logits.float(), dim=1)
    nega_log = -log_probs.gather(1, targets.long().unsqueeze(1)).squeeze(1)  # 정답 칸
    smooth = -log_probs.mean(dim=1)                                          # 전체 평균
    loss = (1.0 - epsilon) * nega_log + epsilon * smooth
    return loss.mean()                    # 배치 평균 스칼라
```

● 2순위: 하이퍼파라미터

- `batch_size = 32`, `learning_rate = 0.0002`, `epochs = 20`. 노트북에 있는 **Task A 점검 셀과 Loss 점검 셀이 통과하는지** 반드시 확인하세요.

P2-2 성능 최적화 · 증강 + Loss/Optimizer/Scheduler + Dropout/TTA

● 1순위: 학습 셀 (점수 영향이 가장 큼)

```
loss_fn = nn.CrossEntropyLoss(label_smoothing=0.2)   # 학습용
eval_fn = nn.CrossEntropyLoss()                       # 평가는 smoothing 없이 → evaluate(model, val_loader, eval_fn)
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate, weight_decay=0.05)
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=learning_rate,
    total_steps=epochs * len(train_loader),           # ← 배치 단위 총 step 수
    pct_start=2 / epochs)                             # warmup 2 epoch
```

```
def train_one_epoch(model, loader, optimizer, loss_fn, scheduler=None):   # 인자 추가
    ...
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)   # backward와 step 사이
        optimizer.step()
        if scheduler is not None:
            scheduler.step()                                      # ← 배치마다 (epoch마다 X)
```

⚠ 호출할 때 `train_one_epoch(..., scheduler)` 로 **scheduler를 꼭 넘기세요**. PDF의 오타 `sceduler`, `evluate`, `filp` 도 고치세요.

● 2순위: `train_transform`

```
T.ToPILImage(),
T.RandomHorizontalFlip(p=0.5),
T.RandomAffine(degrees=10, translate=(0.08, 0.08), scale=(0.9, 1.1), fill=255),
T.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.15),
T.ToTensor(), T.Normalize(mean, std),
T.RandomErasing(p=0.2, scale=(0.02, 0.1), value=0),     # Tensor 전용 → ToTensor 뒤
```

● 3순위: Dropout + flip TTA (모델 셀)

```
def __init__(self, p=0.2, tta=True):  ...  self.dropout = nn.Dropout(p); self.tta = tta
def forward_once(self, x):   # patch_embed → stages 0~3 → mean(dim=(2,3)) → head.norm
    ...;  return self.head.fc(self.dropout(x))         # fc 바로 앞 dropout
def forward(self, x):
    logits = self.forward_once(x)
    if self.tta and not self.training:                 # 평가 때만
        logits = (logits + self.forward_once(torch.flip(x, dims=[3]))) / 2   # 좌우 반전 = W축(3)
    return logits
```

- 하이퍼파라미터: `batch_size = 64`, `learning_rate = 0.0005`, `epochs = 30`.

P3 프롬프트 엔지니어링 · JSON 스키마 출력

● 1순위: `system_prompts()` 에 스키마와 키 넣기

```
schema_text = json.dumps(response_schema, ensure_ascii=False)   # 한글 깨짐 방지
keys_text = ', '.join(response_schema['properties'])
system_prompt = f"""
주어진 녹취에서 계약 정보를 JSON으로 정리하세요.
다음 json schema {schema_text} 를 만족하는 JSON 객체 하나만 출력하세요.
키는 {keys_text} 입니다. 모든 키를 넣고, 확정되지 않았거나 녹취에 없는 값은 null로 씁니다.
""".strip()
```

● 2순위

- f-string 안에 **중괄호를 글자 그대로** 쓰려면 `{{ }}` 로 씁니다. 스키마는 변수로 넣으면 괜찮습니다.
- 프롬프트에 넣을 세 가지: **"JSON만, 설명 문장·코드블록 없이"**, **"모든 키 포함"**, **"모르면 null"**. 날짜나 숫자 형식을 스키마가 정하면 그 형식을 한 번 더 적어 줍니다.
- 본인 GMS API 키로 실제 호출해 **형식 검사 셀이 통과하는지** 확인하세요. 'dummy' 키로는 모의 모드라서 실제 품질을 알 수 없습니다.

● 3순위

- P3는 GPU가 필요 없습니다. **다른 문항의 학습이 도는 동안** 작성하세요.

마지막 30초 체크: ☐ student_id 입력 ☐ 커널 .venv ☐ 수정 금지 셀 그대로 ☐ 실행 결과가 노트북에 남아 있음 ☐ outputs에 .pth와 history.csv ☐ 바꾼 이유를 주석으로